本章路線

這章要回答的核心問題是：系統不確定資源何時釋放時，什麼條件足以造成永久等待，如何用可負擔的策略處理？

前置知識。 process state、mutex 與 semaphore、向量的逐分量比較。

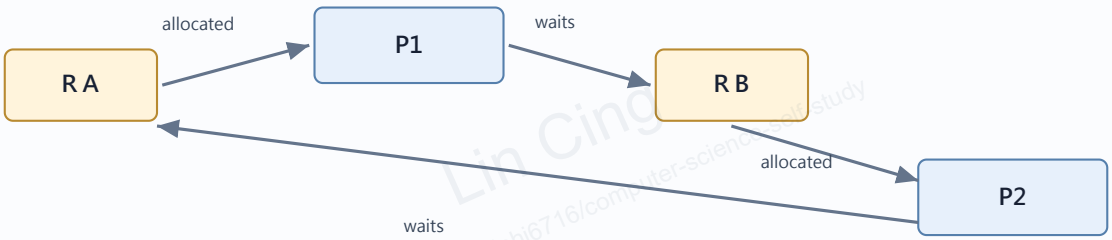
- 證明必要條件不等於充分條件
- 設計破壞 circular wait 的 lock ordering
- 以 Banker's algorithm 檢查 safe state

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

系統模型

- 資源型別有固定 instance 數；process 對資源執行 request、use、release。request 可能立即成功，也可能在資源不足時阻塞。
- 資源配置圖中 process node 指向 resource 表示等待，resource instance 指向 process 表示持有。圖模型只描述等待與持有關係。

Single-instance cycle: A → P1 → B → P2 → A



Two single-instance owners wait for each other and cannot be preempted: deadlocked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資源、狀態與 deadlocked set

resource type 是一組功能等價的 instance。若兩個 printer 可互換，它們屬於同一 type；若位置或功能不可互換，就必須分成不同 type。process 對每個 resource 依序執行 request、use、release。

request 可以立即成功，也可以在 instance 不足時等待。system table 至少保存：

- 每個 type 的 instance 總數。
- 每個 instance 的 free 或 allocated 狀態。
- allocated instance 的 owner。
- 每個 waiting request 的 requester 與 resource type。

一個 process set 處於 deadlocked state，表示集合內每個 process 都在等待一個事件，而該事件只能由集合內的等待者產生，且在既定模型中沒有其他可行途徑解除等待。等待者可以是自己：例如執行緒持有 nonrecursive mutex，又以阻塞方式重複取得同一把鎖，而唯一持有者只能在取得後釋放，就形成自我死結。

兩個已死結的具體狀態

同一 resource type。 三個 CD-RW 共 3 個 instance，P1、P2、P3 各持有 1 個，接著各再請求 1 個。Available=0，且沒有任何 owner 能完成並 release，三者形成 deadlock。

不同 resource types。 系統有 1 個 printer 與 1 個 DVD。P1 持有 DVD 並等 printer，P2 持有 printer 並等 DVD。此狀態滿足：

Process	Allocation	Request
P1	DVD	printer
P2	printer	DVD

資源種類不同不影響 deadlock；判斷依據是 request/hold 關係與是否仍有可完成的 transition。

四個必要條件

條件	在此題中的含義	若持續成立
Mutual exclusion	至少一個 resource 不可同時共享	其他 requester 必須等
Hold and wait	process 已持有至少一個 resource，又等待另一個	等待可能連結兩份 ownership
No preemption	resource 不能被系統強制收回，只能持有者自願 release	外部不能直接拆環
Circular wait	`P0→P1→...→Pn→P0` 的等待鏈	每個參與者的下一步依賴下一人

circular wait 已蘊含 hold and wait，因此四個條件不是彼此邏輯獨立。四者同時成立仍只是 deadlock 的必要條件，不能用來斷言目前已經無限期卡住。

resource-allocation graph 的判準

process node 為圓形，resource type 為矩形，instance 為矩形內圓點。

- request edge `Pi→Rj` 表示 Pi 正在等待 Rj 的 instance。
- assignment edge `Rj→Pi` 表示 Rj 的某個 instance 已分配給 Pi。
- 若 request 立即成功，request edge 立即轉成 assignment edge。
- release 時刪除對應 assignment edge。

以下採一致快照及阻塞請求模型：請求不能取消或逾時，持有資源不可被搶占，等待者取得所需資源前不會提前釋放持有資源；每次請求一個同型實例。

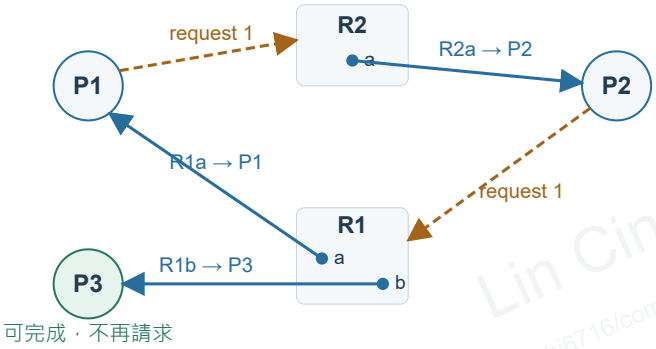
定理。 無 cycle 必無資源死結。若全圖每個 resource type 都只有一個 instance，則存在 cycle **若且唯若**存在死結。即使其他地方有多 instance type，只要某一具體 cycle 上的每個 type 都是單 instance，該 cycle 仍足以證明其參與者死結。

若 cycle 涉及多 instance type，光看這個 cycle 不能判斷是否死結：需檢查其他 instance 的持有者是否能完成並歸還。不能反過來說，多 instance 系統中每個死結都必有一條全由單 instance type 組成的 cycle。

有環，為什麼仍能完成？

R1 有兩個可互換的實例 R1a、R1b，R2 只有 R2a。三個 process 都遵守「取得目前請求後，即可完成並歸還全部持有資源」；P3 不再請求資源，可直接完成。初始沒有空閒實例。

初始狀態：Total=(2,1)，Available=(0,0)



箭頭從誰指向誰？

實線：實例 → 持有者

虛線：等待者 → 資源型別

R1 的兩個點是兩份獨立實例。

環：P1 → R2 → P2 → R1 → P1

P3 不在環上，卻持有另一份 R1。

先讓 P3 完成，就能解開等待。

環上的 P1、P2 目前都在等待，但 R1b 的持有者 P3 並未等待。**有環不等於所有可解環的釋放途徑都被堵住。**

先完成 P3，再依序解除等待

同一個完成順序 P3 → P2 → P1；向量順序為 (R1,R2)

事件	R1a owner	R1b owner	R2a owner	Available
初始：P1、P2 等待	P1	P3	P2	(0, 0)
P3 完成，歸還 R1b	P1	空閒	P2	(1, 0)
授予 P2：R1b	P1	P2	P2	(0, 0)
P2 完成，歸還 R1b、R2a	P1	空閒	空閒	(1, 1)
授予 P1：R2a	P1	空閒	P1	(1, 0)
P1 完成，歸還 R1a、R2a	空閒	空閒	空閒	(2, 1)

授予資源會減少 Available，真正完成並歸還才會增加。只寫完成順序 P3→P2→P1，會省略 P2、P1 取得請求時的兩次扣除；上圖把這些中間狀態一併列出。

Detection 的 Work 是模擬帳，不是即時庫存

向量順序固定為 (R1,R2)。目前尚未滿足的 Request 與既有 Allocation 為：

Process	Allocation	Request	檢查前 Work	模擬完成後 Work
P3	(1,0)	(0,0)	(0,0)	(1,0)
P2	(0,1)	(1,0)	(1,0)	(1,1)
P1	(1,0)	(0,1)	(1,1)	(2,1)

每列先檢查 Request≤Work；若可滿足，再令 Work←Work+Allocation。它把「先授予 Request，之後歸還 Allocation+Request」合成一步，所以淨增加只有 Allocation。Work 中的實例可由已模擬完成者歸還，**不代表那些 process 已真的執行；初始 Available 仍是 (0,0)。**

此處比較的是**目前未滿足的 Request**，不是 Banker's algorithm 的最大剩餘需求 Need。完整 detection 程序見 7.6。

只改 P3 的下一步，就會真的死結

保留圖中的全部持有關係與 P1、P2 的請求，只把 P3 改成「也必須再取得一份 R2 才能完成」，即新增 P3→R2。

初始狀態	P3 的 Request	Work=(0,0) 時可先完成者	結果
原例	(0,0)	P3	完成順序 P3→P2→P1
改例	(0,1)	無	P1、P2、P3 皆死結

改例的 P1 與 P3 都等待 P2 持有的 R2a；P2 等待的 R1，兩份都在這兩個等待者手上。所有實例都困在等待集合內，沒有能先歸還的持有者。此結論假設資源不可搶占、沒有取消或額外實例，也沒有完成前的提前釋放。單純把 P3 暫時移出 CPU 不會改變原例的資源可行性；排程飢餓與資源死結須分開判讀。

lock deadlock 的可重現交錯

thread A 依序取得 `first`、`second`；thread B 依序取得 `second`、`first`。

時刻	動作	持有
t0	A 取得 first	A:first
t1	B 取得 second	A:first; B:second
t2	A 等 second	A 持 first 等 B
t3	B 等 first	B 持 second 等 A

四條件皆成立且沒有非 waiting holder，兩 thread 永久等待。若 A 在 t1 前完成全部工作並 release，這個特定交錯不會發生。deadlock 需要特定相對時序，單次壓力測試不能證明不存在。

四種處理策略的資訊需求

策略	採用的資訊	保證	主要成本
Prevention	只靠限制 request 的協議	至少破壞一個必要條件，理論上不進入 deadlock	低利用率、拒絕 request、rollback
Avoidance	process 事先宣告最大需求	每次配置後仍存在 safe sequence	需要可信 claim，runtime safety test
Detection/recovery	runtime allocation、request 與 snapshot	允許形成 deadlock，之後找出並恢復	資訊維護、偵測延遲、victim 損失
Ignore	不維持 deadlock protocol	不提供一般 deadlock 保證	卡住時由應用、timeout 或重啟處理

第一與第二種都避免「系統進入無法完成全體工作的狀態」，但 prevention 限制 request 形狀，avoidance 則根據未來需求判斷一個目前有空閒的 request 是否仍該等待。

Lock ordering 不變量

- 若每條執行路徑都只依遞增全域順序取得鎖，等待邊永遠由低階指向高階，無法形成嚴格循環。
- 例外路徑、callback、condition wait 與升級鎖也必須遵守順序；只在主路徑排序不足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐一破壞必要條件

Mutual exclusion。 將 resource 改為可共享，例如多人同時讀取 read-only file。mutex 對受保護的可寫資料本質上不可共享，所以這條通常只能局部使用。

Hold and wait。 要求啟動前一次取得全部 resource，或每次 request 前先 release 所有 resource。前者可能讓印表機閒置到程式最後；後者可能破壞尚未持久化的中間資料。

No preemption。 新 request 無法滿足時，收回該 process 現有 resource，待舊與新資源都可重新配置後 restart；或只搶收回應中 waiting process 的資源。只有 state 可保存與重建的資源適合此做法。

Circular wait。 為所有 resource type 指定嚴格順序，只能向較高順序 request。這是實作上最常見的破壞方式，代價是違反順序的 request 必須改成等待或重新設計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

resource ordering 的反證法

令 `F:R→N` 為一對一順序。協議要求 process 持有 `Ri` 時，只能再取得 `Rj`，且 `F(Ri)<F(Rj)`。

假設仍有 circular wait，沿等待邊列 resource 會得到

```
F(R0)<F(R1)<...<F(Rn)<F(R0)
```

由遞移性得到 `F(R0)<F(R0)`，矛盾。因此不可能存在嚴格順序下的 cycle。

動態鎖的實際版本。 兩帳戶 from/to 的管理鎖不能按參數順序取得。先對帳戶 id 排序，再依小 id 到大 id 取得兩鎖；release 可反向或按 API 規則，但新的 acquire 必須維持順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單 instance avoidance 的 claim edge

單 instance graph 額外畫 claim edge `Pi→Rj`，表示 Pi 未來可能 request Rj。request 到來時，claim edge 先轉 request edge；若 grant，再轉 assignment edge。

grant 前要檢查：把提出的 request edge 轉為 assignment edge 後，graph 是否新增 cycle。若無 cycle，保留分配；若有 cycle，Pi 等待，狀態不變。

claim edge 必須在 process 開始前已知，或只能在該 process 尚無 assignment edge 時加入。若未來需求宣告不完整，cycle 檢查無法覆蓋真實 request。cycle detection 可用 adjacency matrix 的 `O(n^2)` 做法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Banker 的資料與 safety 規則

令 n 為 process 數，m 為 resource type 數，`Allocation_i` 是 Pi 現有資源，`Need_i` 是完成前最多還會請求的差額。

```
Need = Max - Allocation
```

safe sequence 的每一步都要求 `Need_i <= Work`。完成 Pi 後：

```
Work = Work + Allocation_i
Finish[i] = true
```

此步驟表示 Pi 能取得它宣告的剩餘需求，完成任務後把全部已配置與新取得資源返回。若所有 Finish 都能變成 true，當下 state 是 safe；否則 unsafe。

Safety algorithm的一般正確性

假設每個process的Max宣告合法、所有resource分量非負、process取得Need後能在有限時間完成並歸還全部資源。Work起始Available；若某Pi的Need_i<=Work，先給它Need，再收回Allocation_i+Need_i，淨變化是Work−Need_i+(Allocation_i+Need_i)=Work+Allocation_i。故完成一步不會減少其他process未來可用的資源，也不把Need平白加進總量。

若存在某安全完成序列，而演算法先挑了序列中不是第一個的可完成Pi，可以先完成Pi再維持其餘原順序、略過原來的Pi位置。其餘每一步面對的可用資源只會更大，不會較小；因此任選一個Need<=Work不會破壞仍存在的安全序列。這說明greedy選擇不需對每個候選分支回溯。

若演算法停止但仍有unfinished，反設原state有安全序列，取該序列中第一個尚未被演算法完成的Pj。它在該序列之前的process都已在演算法的完成集合內；當前Work包含這些Allocation，且可能更多，所以Need_j必<=當前Work，與停止時沒有候選矛盾。故找不到候選而仍有unfinished便unsafe；若全部完成，記錄的順序本身就是安全證書。

resource-request 演算法

Pi提出m維Request_i時，先要求每個分量都是非負整數且維度正確；本節x<=y一律表示每個j都有x[j]<=y[j]。其否定是「存在至少一個分量較大」，不是「每個分量都較大」。

1. 若存在j使Request_i[j]>Need_i[j]，拒絕並視為超過最大宣告。
2. 否則若存在j使Request_i[j]>Available[j]，Pi等待現有資源。
3. 暫時令 `Available-=Request`、`Allocation_i+=Request`、`Need_i-=Request`。
4. 對 trial state 執行 safety algorithm。
5. safe 才真正授予；unsafe 要撤銷 trial 修改，讓 Pi 等待。

反例：Total=(2,1)、唯一Pi的Allocation=(1,0)、Need=(1,1)、Available=(1,1)是合法state。Request=(2,0)在第一維同時超過Need與Available，即應在第1步error；若把向量>誤解為兩維都大，會錯放而得到Available=(−1,1)，已違反非負資源。

完整流程的必要性是：第 3 步是 transaction，不能在 unsafe 後留下部分扣款或部分 allocation。每輪 safety 至多檢查 n 個 process，可做 n 次，因此典型複雜度為 `O(m n^2)`。

Banker T0 完整矩陣

Total=`(10,5,7)`，Available=`(3,3,2)`。

Process	Allocation	Max	Need=Max-A
P0	(0,1,0)	(7,5,3)	(7,4,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

以下的 safety trace 證明 `<P1,P3,P4,P0,P2>` 是 safe sequence：

完成順序	Work 轉移
初始	(3,3,2)
P1	(5,3,2)
P3	(7,4,3)
P4	(7,4,5)
P0	(7,5,5)
P2	(10,5,7)

五種資源：求最小安全庫存

考慮 4 個 process 與 5 種不可搶占、可重複使用的資源。每個 process 取得其剩餘最大需求後，能在有限時間完成並歸還全部資源；向量依資源 A、B、C、D、E 排列。

Process	Allocation	Max	Need = Max − Allocation
P1	(1,0,2,1,1)	(1,1,5,1,2)	(0,1,3,0,1)
P2	(2,0,1,1,0)	(5,2,2,1,0)	(3,2,1,0,0)
P3	(1,1,0,1,0)	(3,1,3,1,0)	(2,0,3,0,0)
P4	(1,1,1,1,0)	(1,1,2,2,1)	(0,0,1,1,1)

已知 Available = (0,0,x,1,1)，x 是非負整數。每個 x 對應資源總量 Total = ΣAllocation + Available = (5,2,4+x,5,2)。求能使狀態安全的最小 x；不同 x 是不同的候選總量，不是憑空改變同一系統的資源帳。

從 Work 的變化找出最小 x

x = 2：每一步先核對 Need ≤ Work

完成者	舊 Work	其 Need		新 Work
P4	(0,0,2,1,1)	(0,0,1,1,1)	→	(1,1,3,2,1)
P1	(1,1,3,2,1)	(0,1,3,0,1)	→	(2,1,5,3,2)
P3	(2,1,5,3,2)	(2,0,3,0,0)	→	(3,2,5,4,2)
P2	(3,2,5,4,2)	(3,2,1,0,0)	→	(5,2,6,5,2)

每步新 Work = 舊 Work + 該 process 的 Allocation

x=0 時，四列 Need 都無法逐分量小於等於初始 Work，沒有第一個候選。x=1 時只有 P4 可先完成；模擬歸還後 Work=(1,1,2,2,1)，P1 缺 C，P2 缺 A、B，P3 缺 A、C，因此無下一個候選。

x=2 時依圖依次核對 Need ≤ Work，得到安全序列 ⟨P4,P1,P3,P2⟩。所以最小的整數 **x=2**。Work 是檢查中的模擬帳；檢查通過並不表示 process 已真正完成或 Available 已變成最後的 Work。

單一資源類型：何時能保證避免死結？

一種資源共有 m 個相同且可重複使用的實例；有限的 n 個 process 各自最多需要 Max_i 個，且 $1 \le Max_i \le m$。已分配資源在完成前不被搶占；資源請求只在可用數不足時阻塞，取得所需資源後 process 能有限時間完成並歸還所有持有量。

對所有尚未完成者，令 $Need_i = Max_i - Allocation_i$。若

$$\sum_{i=1}^{n} Max_i < m + n,$$

則每個合法配置都至少有一個尚未完成者滿足 $Need_i \le Available$。依次讓這些 process 完成，可得到一個安全完成序列。

例：m=10，四個 process 的 Max 均為 3，則 ΣMax=12 < 14；依此模型，無論目前合法配置如何，都有安全序列。此條件**充分但非必要**：若有五人各 Max=3，雖然 ΣMax=15 不小於 m+n=15，但全部尚未持有資源時 Available=10，仍有安全序列。

為什麼每一輪都找得到可完成者？

令目前可用實例數為 v。若所有 n 個尚未完成者都不能滿足最大剩餘需求，則每個 $Need_i > v$；數量為整數，故 $Need_i \ge v+1$。因 $\Sigma Allocation_i = m - v$，得到

$$\sum_i Max_i = \sum_i Allocation_i + \sum_i Need_i \ge (m - v) + n(v + 1) = m + n + (n - 1)v \ge m + n,$$

這與假設矛盾；因此有一人能先完成。這個推理也包含 v>0，不只適用於資源全被占滿。

完成者歸還資源後，從總最大需求中移除其 $Max_i \ge 1$，而人數減一，所以嚴格不等式對剩餘者仍成立；重複此論證即可完成全部 process。這是存在安全序列的資源模型保證；實際系統仍需合理授予請求並讓可執行者持續進展，不能僅靠不等式排除不公平排程造成的飢餓。

P1 request 與 unsafe 拒絕

P1 提出 `Request1=(1,0,2)`：

1. `(1,0,2)<=Need1=(1,2,2)`，未超過 claim。
2. `(1,0,2)<=Available=(3,3,2)`，可做 trial。
3. trial 後 Available=`(2,3,0)`、Allocation1=`(3,0,2)`、Need1=`(0,2,0)`。

Trial safety 可依 `P1→P3→P4→P0→P2` 完成，因此授予此 request。

反例：在上述 trial 後，P0 請求 `(0,2,0)`。資源量足夠，但 trial 後 Work=`(2,1,0)`，沒有任何 P0 到 P4 的 Need 小於等於 Work，safety 失敗，因此 P0 必須等待。

unsafe 不代表此刻已 deadlock。若 process 自願提前釋放資源，仍可能完成；avoidance 的保守保證是「不能保證存在完成序列」，不是「必然永久卡住」。

同樣有空閒，為什麼仍可能拒絕？

以下另用兩類資源 A、B，比較兩次**獨立**試配。總量 (4,3)，初始狀態稱 S，Available=(1,1)。向量一律先 A 後 B。

Process	Allocation	Max	Need=Max−Allocation
P0	(1,0)	(3,2)	(2,2)
P1	(1,1)	(2,2)	(1,1)
P2	(1,1)	(3,2)	(2,1)

逐分量核對：已配置合計 (3,2)，加 Available 得 (4,3)。每列 Allocation+Need=Max，且 Max 不超過 Total。此例假設資源不可搶占、完成前不歸還；取得實際所需資源後可在有限時間完成。

S 是安全狀態：P1 的 Need=(1,1)≤Work=(1,1)，模擬歸還後 Work=(2,2)；再完成 P0 得 (3,2)，最後 P2 得 (4,3)。這個順序只是一份安全證書，不要求 CPU 真的照此排程。

同一份請求，試配給誰有差別

兩次請求各從 S 開始，都是 (1,0)，也都沒有超過該 process 的 Need 或 Available。第二次不接續第一次的試配。

同一個起點，同一份請求，不同的未來

S：Available=(1,1)；兩條分支各自從 S 開始

P0 請求 (1,0)

試配：Available=(0,1)

Allocation0=(2, 0)

Need0=(1, 2)

沒有 Need ≤ (0,1)

unsafe：撤銷全部三項更新

P1 請求 (1,0)

試配：Available=(0,1)

Allocation1=(2, 1)

Need1=(0, 1)

P1 → P0 → P2 可模擬完成

safe：提交試配的狀態

兩邊都只剩 (0,1)，但持有者與剩餘需求不同。

Safety 的完成順序是模擬證書，不是已發生的排程。

安全性不只看剩餘庫存。P1 多取得一份 A 後，只缺一份 B 就能完成；同一份 A 若給 P0，卻沒有人能在最大剩餘需求下先完成。

拒絕 P0：每一列究竟卡在哪裡？

P0 請求 (1,0) 的試配狀態稱 U。Available=(0,1)，只有 P0 改為 Allocation0=(2,0)、Need0=(1,2)。令 Work=(0,1)，所有 Finish 初始均為 false。

候選	試配後 Need	無法開始的具體分量
P0	(1,2)	A：1>0；B：2>1
P1	(1,1)	A：1>0
P2	(2,1)	A：2>0

沒有可先完成者，安全性檢查立即失敗；不是任意選錯第一個 process。實際系統必須回到 S：Available 加回 (1,0)，Allocation0 減去 (1,0)，Need0 加回 (1,0)。其他列與 Max 都不變。

只加回 Available 會把同一份 A 同時計為空閒與已配置。這時 Available+ΣAllocation=(5,3)，已違反 Total=(4,3)。試配是不可被外部觀察到一半的交易；可用鎖保護整段檢查與提交，或在提交時檢查快照版本並重試。

授予 P1：安全性檢查不會真的執行 process

重新從 S 試配 P1 的 (1,0)：Available=(0,1)、Allocation1=(2,1)、Need1=(0,1)。本表只更新 safety 的 Work 與 Finish。

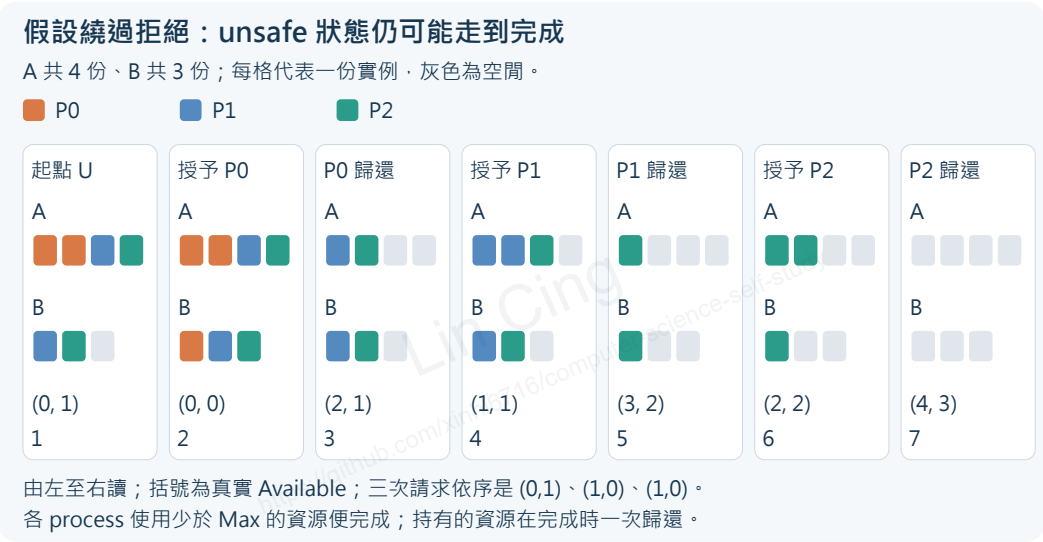
模擬完成者	Need≤舊 Work	加回既有 Allocation	新 Work	Finish，依 P0/P1/P2
初始	—	—	(0,1)	F/F/F
P1	(0,1)≤(0,1)	(2,1)	(2,2)	F/T/F
P0	(2,2)≤(2,2)	(1,0)	(3,2)	T/T/F
P2	(2,1)≤(3,2)	(1,1)	(4,3)	T/T/T

檢查通過後，**提交的是試配狀態，Available 仍為 (0,1)**。不能把最後 Work=(4,3) 寫回 Available，也不能把模擬的 Finish 當作真實終止紀錄。要等 process 真正完成，才會歸還實例。

為何 Work 只加 Allocation？先給 Need、完成時再收回 Allocation+Need，淨變化就是 −Need+(Allocation+Need)=Allocation；Need 不會憑空增加資源總量。

Unsafe 不等於已死結：給出一條真實完成路徑

Banker 會拒絕 U。現在只是反事實比較：假設繞過安全性檢查，真的把 P0 的 (1,0) 授予，停在 U。Max 是上界，不是每個 process 一定會用滿的配額。



這條路徑中，P0 只再要 (0,1)，P1、P2 各再要 (1,0) 就能完成；三者皆未超過 U 的 Need。它證明 U **存在**完成的實際未來，沒有提供應付所有合法未來需求的保證。

同一個 U，也能走向死結

若在 U 中，P0、P1、P2 分別提出整份 Need：(1,2)、(1,1)、(2,1)，每份都合法，但 Available=(0,1) 無法滿足任何一份。三者全都阻塞且仍持有 A；沒有取消、搶占或提前釋放時，便形成死結。

兩條未來使用同一個 Allocation、Max、Available，差在實際請求。安全性檢查使用最大剩餘 Need，要求能保守地保證一個完成順序；detection 則必須看當下真正阻塞的 Request。不能把未來可能請求的 Need 自動當成目前 Request。

Unsafe 狀態沒有安全序列；它仍可能有成功的實際執行。 Safe sequence 是依最大宣告定義的專門概念，不能把任何事後完成順序都叫成它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單 instance：wait-for graph

移除資源節點，保留 process 節點。若存在 request edge `Pi→Rq` 與 assignment edge `Rq→Pj`，合併成 `Pi→Pj`。此邊表示 Pi 等 Pj release Rq。

若每個 resource type 都只有一個 instance，deadlock 存在若且唯若 wait-for graph 有 cycle。cycle 上的每個 process 都不能靠 cycle 外的 instance 自行解除等待。若任何 cycle 經過一個多 instance resource，就必須回到 allocation/request 狀態做可完成性分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多 instance detection 演算法

輸入 `Available`、`Allocation`、`Request`。

```
Work = Available
Finish[i] = (Allocation_i == 0)

while exists i:
    Finish[i] == false and Request_i <= Work:
        Work = Work + Allocation_i
        Finish[i] = true

if any Finish[i] == false:
    Pi is not shown able to complete in this snapshot
```

每次比較採一致快照；標準 reduction 樂觀地假設某 process 的目前 Request 一旦滿足，就能在有限時間完成、歸還全部 Allocation，且完成前不再提出新請求。將沒有持有資源者先標成完成，是因它們不持有其他人所需的資源；不表示這些 process 真的已終止或不在等待。

所有 Finish 為 true 只說明在上述模型與目前 Request 下可構造一個解除等待的模擬順序，不保證真實未來不會再請求或死結。若有 Finish 為 false，它們是此 reduction 無法排除的持有資源者；宣稱它們永久死結還需請求不取消、資源不搶占或提前歸還等假設。複雜度為 $O(m\ n^2)$。

完整 reduction 例

T0 的 Available=$(0,0,0)$。

Process	Allocation	Request
P0	(0,1,0)	(0,0,0)
P1	(2,0,0)	(2,0,2)
P2	(3,0,3)	(0,0,0)
P3	(2,1,1)	(1,0,0)
P4	(0,0,2)	(0,0,2)

完成	Work 加回 Allocation 後
P0	(0,1,0)
P2	(3,1,3)
P3	(5,2,4)
P1	(7,2,4)
P4	(7,2,6)

所有 Finish 為 true，表示在「滿足目前 Request 後有限時間完成且不再請求」的模型中，此快照有可行的解除等待順序；這不是對真實未來永不死結的保證。

P2 多請求後的 deadlocked set

修改 P2 Request=`(0,0,1)`。Work=`(0,0,0)`：

1. P0 Request=`(0,0,0)` 可完成，Work 變成 `(0,1,0)`。
2. 其他 Request 都不能由 Work 滿足。
3. Finish P1、P2、P3、P4 保持 false。

在請求不取消、資源不搶占、不提前釋放的模型下，這個 snapshot 的死結集合為 `{P1,P2,P3,P4}`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

終止 P1 後，重新檢查存活者

前面的P0完成只是偵測演算法的**模擬**，不會真的把資源放回Available。若P0尚未實際完成，終止P1並安全回收其Allocation=(2,0,0)後，真實Available由(0,0,0)變成(2,0,0)，不是(2,1,0)。重新以Work=Available開始，對存活者做reduction：

模擬完成者	當前Request	加回Allocation後的Work
初始，P1已終止	—	(2,0,0)
P0	(0,0,0)	(2,1,0)
P3	(1,0,0)	(4,2,1)
P2	(0,0,1)	(7,2,4)
P4	(0,0,2)	(7,2,6)

每列先驗Request≤Work，完成後只加回原Allocation。在目前 Request 獲得滿足後各存活者都能有限時間完成、無新請求的假設下，這個序列證明存活者有可行完成順序；只有它們真的執行並歸還資源，真實Available才依序增加。若P0早已實際完成，終止P1後才可直接由Available=(2,1,0)開始。

偵測頻率越高，越早找出最接近最後等待的 requester，但執行成本也越高；固定 interval 可能一次看到多個 cycle。

終止 process 的恢復

全部終止。 一次 abort 所有 deadlocked processes，保證拆環，但損失全部未完成工作。

逐一終止。 每次 abort 一位 victim，回收其全部 resource，再重跑 detection。未終止者若因資源返回而可進展，就停止犧牲。

victim 的選擇是 policy：

- priority 與互動程度。
- 已執行時間、剩餘估計時間。
- 持有與需要的 resource 數量。
- 資源是否容易 reclaim。
- 需要終止多少個 process 才能拆環。

若 process 正在修改 file 或驅動裝置，直接 abort 可能留下外部半完成狀態。恢復設計要把 rollback、journal 或 device reset 納入成本，不能只把 lock 視為可回收物件。

resource preemption 與 starvation

resource preemption 要回答三個獨立問題：

1. **Victim。** 要收回哪位 process 的哪些 resource，成本如何計算。
2. **Rollback。** 被收回者要回到哪個一致狀態。最容易是全部 abort 後 restart；較省成本的是回到 checkpoint，但需要保存足夠 undo state。
3. **Starvation。** 同一 process 不能永遠因 cost 最低而被選為 victim。

`cost_i = execution_i + rollback_count_i × k`（k>0）會提高反覆回滾者的相對成本，是減少重複選取的啟發式。只有在競爭集合與其他候選成本受適當限制時，才可推導它何時不再最低；任意新到達或成本變動下，不能據此保證公平。明定每個 process 最多連續被選 r 次（r 有限）可限制連續受害次數，但仍須交代沒有其他可選 victim 時的恢復策略；A、B 仍可能交替無限次被選，因此此局部限制不能保證全體最終完成。有限等待或完成保證須由完整 workload 與選取規則推導。

deadlock 判斷的完整決策表

你觀察到什麼	可以先得到什麼	還需要什麼
四必要條件成立	deadlock 可能	是否存在無可進展 transition
single-instance wait-for graph 無 cycle	未 deadlocked	不需再做 cycle 充分性判斷
single-instance graph 有 cycle	deadlocked	確認所有 cycle resource 都是單 instance
multi-instance graph 有 cycle	deadlock 可能	availability/reduction 或 safety 分析
unsafe state	avoidance 不能提供完成保證	實際 request 可能尚未造成 deadlock
某 process 長期未執行但系統持續前進	starvation 可能	公平 policy 與等待上界

deadlock 是等待集合沒有可完成的內部 transition。starvation 是系統仍有 transition 與進展，但某個體沒有 finite wait bound。兩者可以同時存在，也可以各自獨立發生。

本章收束

- 必要條件不能單獨證明充分
- avoidance 依賴最大需求宣告
- 檢測、恢復與資料一致性必須一起設計

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study